

Goti Striker — Gameplay Area Specification

Dimensional reference for building environments, maps and surrounding set dressing.
Unity 6000.6.0f1 · URP 17.6.0 · Generated 16 September 2026 · Repo `admin-xtinex/goti_striker`

1. Scope and how to read this

Every figure here was read out of the project source, not estimated. Each table names the file it came from so you can re-check it after a change. Values marked **derived** are calculated from those constants; the calculation is shown so you can audit it.

The gameplay prefab is **map-independent**. It carries its own collision and needs no art from your map. Your job as a map author is to build visuals that agree with the numbers below and to stay out of the reserved volumes.

2. Coordinate system

Standard Unity left-handed, Y-up. All coordinates are in the gameplay prefab's **local space**. Place the prefab at world origin with no rotation and local equals world.

Axis	Direction	Meaning in this game
+X	Right	Lane width. Centreline is <code>X = 0</code> .
+Y	Up	Height. Playing surface top is <code>Y = 0</code> .
+Z	Forward	Lane length. Player shoots from low Z toward high Z.

Unit = 1 metre. Do not author your map at a different scale and rescale the prefab — the marble radius, pit rim and shot forces are all absolute, and scaling the prefab breaks the physics tuning.

3. Master dimensions

3.1 Collision surface — the physical floor

Property	X	Y	Z
Extent	-10.0 ... +10.0	-0.50 ... 0.00	-8.5 ... +37.5
Size	20.0 m	0.50 m (slab)	46.0 m

Source: `BuildMapReadyKit.cs` — `HalfWidth = 10`, `SurfaceTopY = 0`,
`SurfaceThickness = 0.5`, `ZMin = -8.5`, `ZMax = 37.5`.

Built as **tiled boxes, not one slab**, so the three pit openings stay clear. Never replace it with a single unbroken collider — that seals the pits and the game stops working.

3.2 Playable corridor — where the marble is actually held

The marble is clamped by script well inside the floor, and the corridor **widens toward the far end**.

Position	Half-width X	Full width	Z
Near end	±5.15	10.30 m	−8.5
Far end	±6.65	13.30 m	+37.5
Length	—	—	46.0 m

Source: `MarbleController.cs:182-184` . The gap between this corridor and the 20 m floor is deliberate margin — it is what prevents a marble reaching an unsupported edge and falling.

Known off-by-one: the taper interpolates over `46.5` while the Z span is `46.0` , so the far half-width resolves to **6.634**, not 6.65. 16 mm, no gameplay effect, documented rather than silently rounded.

3.3 Pits

Property	Value
Positions (Z)	3.0, 16.5, 31.0
Lateral position	<code>x = 0</code> — all three on the centreline
Spacing	13.5 m (pit 1→2), 14.5 m (pit 2→3) — uneven by design
Rim radius	0.52 m (1.04 m across)
Basin depth	approx. −0.08 ... −0.18 m below surface
Collision opening	0.72 × 0.72 m square gap in the floor
Capture trigger	1.50 m radius sphere, submerged

Sources: `BuildMapReadyKit.cs` (`PitZ` , `PitGapHalf = 0.36`), `PitZone.cs` (rim, trigger).

The square opening sits fully inside the rim disc ($0.36 \times \sqrt{2} = 0.509 < 0.52$), so the saucer mesh always floors the gap: a marble can enter but never drop through.

Do not normalise the pit spacing. The shot power curve is tuned to 13.5 and 14.5 m. Making them equal changes the difficulty of every shot in the game.

3.4 Boundary walls

Property	Value
----------	-------

Height	1.50 m ($Y = 0.00 \dots 1.50$)
Thickness	0.30 m
Left / Right	$X = \pm 10.15$ (centre of the 0.3 m box)
Near / Far	$Z = -8.65$ / $Z = +37.65$

Invisible and always solid. Because the script clamp (§3.2) is tighter than the walls, the clamp is what a marble meets first; the walls are a backstop.

up to ≈ 7.0	camera may reach this height	keep clear
above 7.0	nothing	free — sky, canopy, distant architecture

Flight apex — derivation

Step	Value	Source
Max launch force	32.0 N	<code>SwipeLaunchController._maxLaunchForce</code>
Easy-mode multiplier	$\times 1.15$	<code>GameDifficultyConfig.LaunchForceMultiplier</code>
Effective impulse	36.8 N·s	derived
Marble mass	1.0 kg	<code>MarbleController._mass</code>
Launch speed	36.8 m/s	<code>ForceMode.Impulse</code> , $v = F/m$
Max loft pitch	0.2475	<code>lerp(0.20, 0.55, 1.0) $\times$ 0.45</code>
Normalised vertical	0.2402	derived
Vertical speed	8.84 m/s	derived
Apex above surface	≈ 4.15 m	$v^2/2g + 0.16$

Reserve 5 m of clear height over the whole corridor for safety.

6. Camera envelope — read this before placing any prop

The camera is a fixed-offset follow rig that **orbits freely 360° around the marble**.

Property	Value	Source
Distance behind marble	4.8 m	<code>SmoothFollowCamera._distance</code>
Height above marble	2.6 m	<code>SmoothFollowCamera._height</code>
Look-at offset	+0.45 m	<code>SmoothFollowCamera._lookAtHeightOffset</code>
Smoothing	0.22 s	<code>SmoothFollowCamera._smoothTime</code>
Orbit	full 360°, player-controlled	<code>_orbitSensitivity</code>

The camera has NO occlusion avoidance

`SmoothFollowCamera.LateUpdate()` computes `desiredPosition = target.position - aimDir × 4.8 + up × 2.6` and contains **no Physics. call of any kind** — no raycast, no sphere-cast, no collision push-in.

Consequences you must design around:

1. Solid geometry inside the camera volume will **hard-occlude the view**. The camera will not pull in to avoid it.
2. The camera **passes straight through walls and props**. Anything it clips will show its backfaces or vanish.

Keep-clear volume

Because the camera can be anywhere within 4.8 m of the marble, and the marble can be anywhere in the corridor:

Axis	Keep clear
X	-11.5 ... +11.5
Y	0 ... 7.0
Z	-13.5 ... +42.5

Nothing opaque inside that box. Set dressing starts **outside** it — which is 1.5 m beyond the collision floor on each side, 5 m beyond each end.

7. Zone guide — what you may build, and where

Zone	Volume	You may place
------	--------	---------------

The lane	$X \pm 10, Y = 0$	Ground visuals only. Flat, top at exactly $Y = 0$. No colliders.
Pit locations	$r \geq 0.52$ at $Z\ 3.0 / 16.5 / 31.0$	Leave clear, or cut openings ≥ 1.04 m across. Rim decoration outside $r = 0.52$.
Under the lane	$Y < -0.50$	Anything — bedrock, stilts, water, a drop to a valley.
The near apron	$Z -13.5 \dots -8.5$	Visuals only, below $Y = 0$ or outside $X \pm 11.5$.
Inner margin	$X \pm 10 \dots \pm 11.5$	Flat ground visuals only. Nothing tall — the camera passes through here.
Surrounding set	outside $X \pm 11.5, Z -13.5 \dots 42.5$	Free. Buildings, trees, crowds, terrain, fences, hills.
Overhead	$Y > 7.0$	Free. Canopy, banners, sky, birds — nothing below 7 m.

Layers

The project defines these; assign your map geometry accordingly.

Index	Layer	Use
0	Default	Decorative map geometry with no collider
6	Marble	Marbles (prefab-owned)
7	GameplayGround	Collision surface (prefab-owned)
8	Obstacle	Boundary walls (prefab-owned)
9	PitTrigger	Pit capture triggers (prefab-owned)

Map art should sit on **Default** with no collider. Do not put map geometry on layers 6–9 — the gameplay code queries those.

8. Steps to add a map in Unity

Step 1 — Open the map scene

Assets/_Project/Scenes/SC_Village_Graphics_Test.unity

This is the scene the gameplay prefab is instantiated into. The gameplay prefab itself is Assets/_Project/GameplayKit/Prefabs/GotiStriker_GameplayCore.prefab .

Work in the scene, never in the prefab. The prefab is map-independent on purpose. Any ground, lighting or dressing you add belongs to the scene so that swapping maps later never touches gameplay.

Step 2 — Remove the temporary soil placeholder

In the Hierarchy, delete the root object:

TemporaryGround_PlaceholderMap

It is purely visual (zero colliders), so deleting it cannot break physics. It contains the soil lane, two aprons and a backing plane under the pits. Re-create it any time with **Tools** → **Pit Striker** → **Gameplay Placement** → **Add Temporary Soil Ground To Map**.

Step 3 — Import your map art

Drop your environment package or model into Assets/_Project/Art/ . Free Asset Store environments work fine. Keep it out of GameplayKit/ — that folder is gameplay only.

Step 4 — Position the ground plane

Align the art so that:

Requirement	Value
Ground top surface	exactly <code>Y = 0</code>
Covers at least	<code>X -10 ... +10</code> , <code>Z -8.5 ... +37.5</code>
Scale	1 unit = 1 metre, prefab left at scale 1

If your ground model has thickness, offset it down so the **walkable top face** lands on `Y = 0` — not its pivot.

Step 5 — Clear the pits

Cut holes, or leave gaps, at $(0, 3.0)$, $(0, 16.5)$, $(0, 31.0)$ — each at least **1.04 m** across. If your ground mesh cannot be cut, place three separate ground pieces between the pits.

If holes end up wider than the 1.04 m saucer, add a backing plane below (around $Y = -0.26$) or you will see **sky through the ground** around each pit.

Step 6 — Strip colliders from the art

Select all imported map geometry and **remove every collider**. The gameplay prefab supplies 100% of the physics. A stray **MeshCollider** on decorative ground will fight the tiled collision surface and produce jitter or seal a pit.

Checklist: map art renderers > 0, map art colliders = 0

Step 7 — Respect the camera volume

Re-read §6. Move anything tall out of $X \pm 11.5$ / $Z -13.5...42.5$ / $Y 0...7$. This is the single most common cause of "the camera goes inside a building".

Step 8 — Light it

Lighting is scene-owned. The prefab has none. Set up your directional light, sky and bake as you like. Keep the surface readable — the marble is small (0.32 m) and the player must track it at 20+ m.

Step 9 — Validate in the Editor

Run the bounds validator, which checks spawn points against actual collider volumes in collider-local space:

Tools → Pit Striker → Gameplay Placement → Build Map-Ready Kit

Then enter Play mode and confirm:

#	Check	Expected
1	Marble spawns and rests	sits at $Y \approx 0.16$, does not sink or fall
2	Full-power shot down the lane	travels to Z 37 without falling through
3	Each of the three pits	marble can enter and be captured
4	Lane edges	marble is held inside the corridor
5	Camera orbit, full 360°, at both ends	never enters geometry, never occluded
6	Look down at each pit	no sky visible through the ground

Step 10 — Commit

Commit the scene and the art. Do not commit anything under `Keystore/` or other secrets — the repo's `.gitignore` covers the signing keystore at line 66.

9. Quick reference card

SURFACE TOP	Y = 0	
FLOOR	X -10 .. +10	Z -8.5 .. +37.5 (20.0 x 46.0 m)
CORRIDOR (near)	X -5.15 .. +5.15	(10.30 m wide)
CORRIDOR (far)	X -6.65 .. +6.65	(13.30 m wide)
PITS	X 0	Z 3.0 / 16.5 / 31.0 rim r 0.52
WALLS	X +/-10.15, Z -8.65 / +37.65	h 1.5 t 0.3
TEE / SPAWN	(0, 0.3, -6.0)	
MARBLE	r 0.16 (0.32 diameter)	mass 1 kg
FLIGHT APEX	~4.15 m	-> reserve 5 m clear
CAMERA	4.8 m back, 2.6 m up, orbits 360,	NO occlusion avoidance
CAMERA KEEP-CLEAR	X +/-11.5 Y 0..7.0	Z -13.5..+42.5
LAYERS	7 GameplayGround	8 Obstacle 9 PitTrigger 6 Marble

Sources: `BuildMapReadyKit.cs`, `MarbleController.cs`, `PitZone.cs`, `SmoothFollowCamera.cs`, `ShotModeConfig.cs`, `SwipeLaunchController.cs`, `GameDifficultyConfig.cs`, `TurnManager.cs`, `AddTemporaryGround.cs`, `TagManager.asset`.